

Aula 2: Complexidade e reduções em bioinformática

Luís Felipe

UFF

18 de Agosto de 2026

Pequena mudança de modelo pode mudar a complexidade

Pergunta

Como reconhecer formalmente a dificuldade e transferi-la de um problema para outro?

Exemplos de formulações **tratáveis**:

- alinhamento de duas sequências
- distância DCJ entre dois genomas
- inversões em permutações com sinais
- estrutura de RNA sem pseudonós

Exemplos de formulações **intratáveis**:

- alinhamento de um número arbitrário de sequências
- mediana DCJ de três genomas
- inversões em permutações sem sinais
- modelos gerais que admitem pseudonós

Objetivo: Rever conceitos de complexidade computacional e visitar alguns resultados conhecidos.

A classe de complexidade pertence à formulação precisa

Um problema computacional deve especificar

- a **instância** e sua codificação;
- quais objetos contam como **soluções**;
- a **pergunta** ou função objetivo;
- o **tamanho da entrada**;
- operações, custos e restrições permitidos.

Consequência

Não basta dizer “alinhamento”, “folding” ou “rearranjo”. A complexidade pode depender do número de sequências, do sistema de pontuação, das operações permitidas e de outros parâmetros do modelo.

Eficiência

- Um **algoritmo eficiente** é aquele cuja complexidade é um polinômio no tamanho da sua entrada. (Edwards, 1964)
- Um **problema eficiente** é aquele que admite algoritmo eficiente.
- Problemas eficientes são denominados **tratáveis** e os não eficientes são **intratáveis**.
- Notação para problemas: $\Pi(D, Q)$
 - ▶ D = Conjunto de dados de Π
 - ▶ Q = Questão que Π deve resolver

OBS.: A existência de um algoritmo não eficiente para um problema não implica na intratabilidade do problema em questão.

Tipos de Problemas

- Um **Problema de Otimização** é um problema cuja questão Q consiste em **otimizar** (**minimizar** ou **maximizar**) a solução, entre todas as possíveis, levando em consideração o conjunto D .
- Em um **Problema de Decisão** a questão Q consiste em uma pergunta cuja resposta em **Sim** ou **Não**.

Dado um problema de otimização, podemos formular um problema de **decisão associado**.

O problema de decisão associado ao de otimização possui o mesmo conjunto D de dados do problema de otimização, **além de um valor k** .

A questão consiste em saber se a solução é $\leq k$ ou $\geq k$, para um problema de **minimização** ou **maximização**, respectivamente.

Otimização e decisão respondem perguntas diferentes

Exemplo:

Otimização

Encontrar o **menor conjunto de primers** que cobre todos os alvos.

Qual é o mínimo?

Decisão

Dados os candidatos, os alvos e k , existem **no máximo k primers** que cobrem todos os alvos?

A resposta é sim ou não?

Classes $\mathcal{P}$ e $\mathcal{NP}$ de problemas

$\mathcal{P}$:

- A **classe** $\mathcal{P}$ de problemas (de decisão) corresponde exatamente aos problemas **tratáveis**.

Certificado:

- Um **certificado** para um problema de decisão é uma **evidência** que, quando validada, atesta a solução do problema.
- Assim sendo, o certificado pode ser para a resposta **Sim** ou **Não**.
- Para validar um certificado, aplica-se um **algoritmo de validação**.

$\mathcal{NP}$:

- A **classe** $\mathcal{NP}$ de problemas de decisão corresponde aos problemas Π que admitem certificados para a resposta **Sim**, que possam ser validados através de um algoritmo eficiente em função da entrada de Π .

Consequentemente,

$\mathcal{P} \subseteq \mathcal{NP}$. Porém $\mathcal{P} = \mathcal{NP}$ ou $\mathcal{P} \neq \mathcal{NP}$?

Transformações polinomiais



- Se a função f for computável em tempo polinomial no tamanho de D_1 , então a **transformação** é dita **polinomial**.
- Se for conhecido um algoritmo para Π_2 , então a transformação polinomial conduz a um algoritmo polinomial para resolver Π_1 .
- Se Π_1 for polinomialmente transformável em Π_2 , usamos a notação $\Pi_1 \propto \Pi_2$.
- Em outras palavras, se $\Pi_1 \propto \Pi_2$, então uma entrada **arbitrária** do problema Π_1 pode ser resolvida usando um **número polinomial de passos na transformação** mais um **número polinomial de chamadas ao algoritmo** que resolva o problema Π_2 .

Subsequência comum e distância por indels

LCS

$LCS(x, y)$ é o comprimento da **maior sequência comum** obtida de x e de y pela remoção de símbolos, sem alterar a ordem dos restantes.

Exemplo

Considere

$$x = \text{ACGT} \quad \text{e} \quad y = \text{AGCT}.$$

Duas subsequências comuns máximas são

$$\text{AGT} \quad \text{e} \quad \text{ACT}.$$

Logo, $LCS(x, y) = 3$.

Uma transformação ótima por indels é

$$\text{ACGT} \xrightarrow{\text{deletar C}} \text{AGT} \xrightarrow{\text{inserir C}} \text{AGCT}.$$

Portanto, $d_{ID}(x, y) = 2$.

Distância por indels

$d_{ID}(x, y)$ é o **menor número** de **inserções** e **deleções** unitárias que transforma x em y .

A identidade entre LCS e indels fornece a redução

Seja z uma LCS de x e y , com

$$\ell = |z| = \text{LCS}(x, y).$$

Transformação guiada por uma LCS

Para transformar x em y :

1. deletamos de x os símbolos que não pertencem à ocorrência escolhida de z : são $|x| - \ell$ deleções;
2. inserimos em z os símbolos que faltam para obter y : são $|y| - \ell$ inserções.

Assim, usamos

$$(|x| - \ell) + (|y| - \ell) = |x| + |y| - 2\ell$$

operações.

Reciprocamente, os símbolos preservados por qualquer sequência de inserções e deleções formam uma subsequência comum. Portanto, nenhuma transformação pode usar menos operações.

Identidade

$$d_{ID}(x, y) = |x| + |y| - 2\text{LCS}(x, y).$$

A identidade transforma uma instância na outra

Considere a instância

$$(x, y, k)$$

do problema de decidir se

$$\text{LCS}(x, y) \geq k.$$

Construímos a instância

$$(x, y, b)$$

do problema de distância por indels, onde

$$b = |x| + |y| - 2k.$$

Equivalência

$$\text{LCS}(x, y) \geq k \iff d_{ID}(x, y) \leq |x| + |y| - 2k.$$

A transformação apenas calcula b e, portanto, é realizada em tempo polinomial.

Consequência

LCS de duas strings pode ser calculada por Programação Dinâmica em tempo polinomial (**Algoritmo de caminho máximo numa DAG desde a origem**, Aula 10 APA/ASA).

O que a redução mostrou

- uma relação exata entre duas funções objetivo;
- como traduzir limites de uma versão de decisão;
- como uma solução para um problema pode resolver o outro.

Conclusão

Tanto LCS de duas strings quanto distância por inserções e deleções pode ser calculada por programação dinâmica em tempo polinomial.

Problemas $\mathcal{NP}$ -completos

$\mathcal{NP}$ -completude:

Um problema Π é $\mathcal{NP}$ -completo quando:

1. $\Pi \in \mathcal{NP}$, e
2. $\Pi' \propto \Pi, \forall \Pi' \in \mathcal{NP}$

- Se um problema Π satisfaz 2., então Π é $\mathcal{NP}$ -difícil.

Pergunta: Como mostrar que um problema é $\mathcal{NP}$ -completo?

Reduções de $\mathcal{NP}$ -completude:

Lema. Sejam Π_1, Π_2 problemas tais que $\Pi_1, \Pi_2 \in \mathcal{NP}$. Então, se Π_1 é $\mathcal{NP}$ -completo e $\Pi_1 \propto \Pi_2$, então Π_2 é $\mathcal{NP}$ -completo.

Prova: Π_1 é $\mathcal{NP}$ -completo, então $\Pi' \propto \Pi_1, \forall \Pi' \in \mathcal{NP}$. Como $\Pi_1 \propto \Pi_2$, por transitividade, $\Pi' \propto \Pi_2$. Logo, Π_2 é $\mathcal{NP}$ -completo. $\square$

Como elaborar uma $\mathcal{NP}$ -completude??

Para provar que um problema Π é $\mathcal{NP}$ -completo, fazemos:

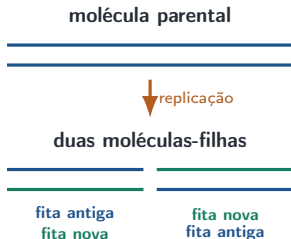
1. Mostramos que $\Pi \in \mathcal{NP}$
2. Escolhemos um problema Π' que seja $\mathcal{NP}$ -completo e mostramos que $\Pi' \propto \Pi$

A replicação celular copia o DNA antes da divisão

Replicação do DNA

Processo celular em que a dupla hélice se abre e cada fita antiga serve como **molde** para a síntese de uma nova fita complementar.

- o pareamento $A-T$ e $C-G$ determina a nova sequência;
- a síntese começa em regiões chamadas **origens de replicação**;
- a abertura do DNA forma **forquilhas de replicação**.



Replicação semiconservativa

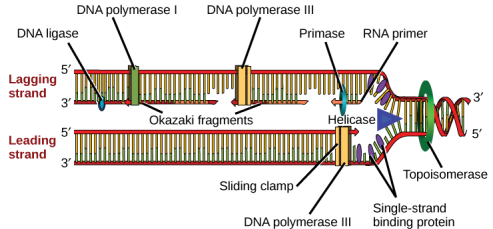
Cada molécula produzida reúne uma fita antiga e uma fita recém-sintetizada.

As duas novas fitas são sintetizadas simultaneamente

Direção da síntese

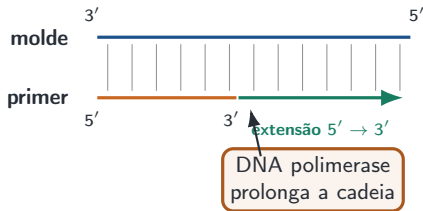
A **DNA polimerase** percorre a fita molde no sentido $3' \rightarrow 5'$ e acrescenta nucleotídeos à nova fita no sentido $5' \rightarrow 3'$.

- **fita líder**: síntese contínua na direção da forquilha;
- **fita tardia**: síntese em fragmentos de Okazaki, orientados para longe da forquilha;
- moléculas de DNA polimerase atuam simultaneamente nas duas fitas como parte do complexo de replicação.



Na figura, a forquilha avança para a direita; a fita líder cresce continuamente e a tardia forma fragmentos de Okazaki.

O primer fornece a extremidade 3' que inicia a síntese



Primer

Pequena cadeia de nucleotídeos complementar ao molde que oferece uma **extremidade 3' livre**. A DNA polimerase prolonga essa cadeia.

- na célula, a **primase** produz primers de RNA;
- a fita líder começa a partir de um primer;
- cada fragmento de Okazaki começa a partir de um novo primer;
- os primers são substituídos por DNA e a **ligase** une os fragmentos.

Na PCR, dois primers delimitam uma sequência-alvo

Sequência-alvo

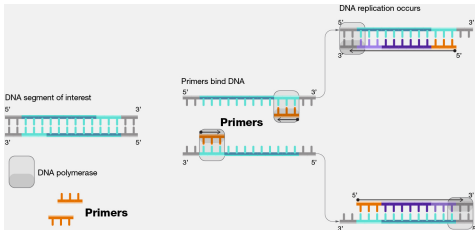
Região do DNA escolhida para ser amplificada e analisada: por exemplo, um gene, um locus com um SNP ou uma sequência característica de um patógeno.

Reação em Cadeia da Polimerase (PCR)

Técnica de laboratório que repete três etapas: separação das fitas, ligação dos primers e extensão pela DNA polimerase. A repetição produz **muitas cópias do alvo** (n repetições, 2^n cópias).

Par de primers

Duas pequenas fitas sintéticas de DNA. Cada uma se liga a uma fita molde, em um dos lados da sequência-alvo.

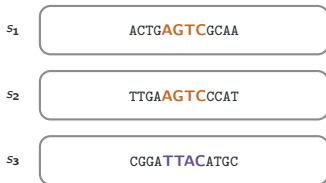


Os primers, em laranja, oferecem extremidades 3' voltadas para o interior do alvo; as novas fitas aparecem em roxo.

Amplicon e aplicações

O **amplicon** é o fragmento entre os dois primers. Suas cópias podem ser usadas em detecção, genotipagem e sequenciamento.

Um primer pode cobrir vários alvos



Os trechos coloridos são sítios de ligação. O primer sintético possui a sequência complementar ao sítio.

Cobertura do exemplo

p_1 , complementar a AGTC, atende s_1 e s_2 ; p_2 , complementar a TTAC, atende s_3 :

$$A(p_1) = \{s_1, s_2\}, \quad A(p_2) = \{s_3\}.$$

Alvo e cobertura

Cada s_i é uma região de DNA escolhida para amplificação, por exemplo, um membro conhecido de uma família de genes. Um primer p **cobre** s_i quando encontra nela um sítio de ligação compatível.

Do primer à cópia

O primer liga-se ao sítio; a DNA polimerase parte de sua extremidade 3' e usa o DNA-alvo como molde para copiar a região adjacente. Assim, um primer curto inicia a síntese de uma região longa.

Seleção no experimento

Um primer compartilhado pode atender vários alvos. **Buscamos o menor repertório que cobre todos eles**, reduzindo os oligonucleotídeos a sintetizar e manipular. Na PCR completa, primers compatíveis são combinados em pares que delimitam cada amplicon.

Optimal Primer Cover minimiza o conjunto de primers

Optimal Primer Cover – OPC

Considere um conjunto de sequências-alvo $S = \{s_1, \dots, s_n\}$ e um inteiro k . Na convenção do modelo, cada primer candidato é representado por uma substring de comprimento pelo menos k . O primer p cobre s_i quando $p \sqsubseteq s_i$

($x \sqsubseteq y$ significa que x está contido como substring de y , consecutivamente).

Para cada primer candidato, defina

$$A(p) = \{s_i \in S : p \sqsubseteq s_i\}.$$

Otimização

Encontrar um conjunto P' de menor tamanho tal que

$$\bigcup_{p \in P'} A(p) = S.$$

Decisão

Dados S , k e um limite b , existe um conjunto P' tal que

$$|P'| \leq b \quad \text{e} \quad \bigcup_{p \in P'} A(p) = S?$$

Qual é o número mínimo de primers?

A resposta é sim ou não?

NP-completude de Optimal Primer Cover

Teorema

A versão de decisão do **Optimal Primer Cover** é **NP-completa**. A versão de otimização é **NP-difícil**.

Pertencimento a NP

Um certificado é um conjunto P' com no máximo b primers. Para cada sequência s_i , verificamos em tempo polinomial se algum $p \in P'$ ocorre em s_i .

NP-dificuldade

Pearson et al. (1995) apresentam uma redução polinomial de **Set Cover** para **Optimal Primer Cover**.

A redução de Set Cover estabelece a NP-completude

Considere uma instância de decisão de Set Cover: universo T , família $\mathcal{M} = \{M_1, \dots, M_m\}$ e limite b .

Set Cover		Optimal Primer Cover
elemento $x \in T$	$\longleftrightarrow$	sequência-alvo s_x
conjunto $M_j \in \mathcal{M}$	$\longleftrightarrow$	palavra de primer u_j
pertinência $x \in M_j$	$\longleftrightarrow$	u_j ocorre como substring de s_x
escolher até b	$\longleftrightarrow$	escolher até b primers
conjuntos		

Construção

O artigo constrói, em tempo polinomial, as sequências s_x usando substrings exclusivas para codificar as pertinências: $u_j \sqsubseteq s_x$ exatamente quando $x \in M_j$.

Equivalência

$$\exists \mathcal{M}' \subseteq \mathcal{M}, |\mathcal{M}'| \leq b, \bigcup_{M \in \mathcal{M}'} M = T \iff \exists P', |P'| \leq b, \bigcup_{p \in P'} A(p) = S.$$

Assim, OPC é NP-difícil; como pertence a NP, sua versão de decisão é NP-completa.

Variação pode mudar complexidade

Em strings, o número de sequências pode ser decisivo:

Problema	Formulação tratável	Formulação difícil
LCS	duas strings: programação dinâmica, tempo polinomial	número arbitrário de strings como parte da entrada: decisão NP-completa, inclusive em alfabetos pequenos
alinhamento	duas sequências com pontuação aditiva usual: programação dinâmica	alinhamento múltiplo com número de sequências não limitado: NP-completo em formulações clássicas
superstring	duas strings: escolha simples da melhor sobreposição	menor superstring comum para um conjunto de strings: NP-difícil

Leia as hipóteses

O resultado depende de quais quantidades fazem parte da entrada e do modelo de pontuação. “Alinhamento múltiplo é NP-completo” sem essas condições é uma afirmação incompleta.

Multi-LCS: definição computacional

Subsequência

Para $s = s_1 s_2 \cdots s_m$ e $z = z_1 z_2 \cdots z_\ell$, escrevemos $z \preceq s$ quando existem índices

$$1 \leq p_1 < p_2 < \cdots < p_\ell \leq m \quad \text{tais que} \quad z_j = s_{p_j} \text{ para todo } j.$$

Portanto, z é obtida apagando símbolos de s , sem alterar a ordem dos símbolos preservados.

Multi-LCS – otimização

Instância: um alfabeto finito Σ e strings $s_1, \dots, s_d \in \Sigma^*$, com d fazendo parte da entrada.

Objetivo: encontrar uma string de comprimento máximo no conjunto

$$\{z \in \Sigma^* : z \preceq s_i \text{ para todo } i \in \{1, \dots, d\}\}.$$

Multi-LCS – decisão

Dados $s_1, \dots, s_d$ e um inteiro L , existe $z \in \Sigma^*$ tal que

$$|z| \geq L \quad \text{e} \quad z \preceq s_i \text{ para todo } i?$$

A dimensão da PD cresce com o número de strings

Considere d strings $s_1, \dots, s_d$, com $|s_i| = n_i$.

d fixo

A generalização da programação dinâmica usa um estado para cada vetor de prefixos

$$(i_1, \dots, i_d), \quad 0 \leq i_j \leq n_j.$$

O número de estados é

$$\prod_{j=1}^d (n_j + 1).$$

Para $n_j \leq n$, o tempo é $O(dn^d)$:
polinomial quando d é constante.

d faz parte da entrada

O expoente d passa a depender da própria instância. Assim, n^d não é limitado por N^c para uma constante c , onde N é o tamanho da entrada.

A pergunta de complexidade

Existe um algoritmo polinomial diferente ou o problema de decisão é NP-completo?

A versão de decisão de Multi-LCS pertence a NP

Considere uma instância $(s_1, \dots, s_d, L)$ da versão de decisão.

Certificado

Uma string z com $|z| \geq L$.

O certificado tem tamanho polinomial:
basta considerar $|z| \leq \min_i |s_i|$.

Verificação

Para cada s_i , um único percurso da esquerda para a direita verifica se $z \preceq s_i$.

O tempo total é

$$O\left(\sum_{i=1}^d |s_i|\right).$$

Consequência

O verificador não precisa provar que z é ótima: basta confirmar que ela é comum e que $|z| \geq L$. Portanto, Multi-LCS está em NP.

A NP-completude já ocorre no alfabeto binário

Teorema – Maier, 1978

Multi-LCS, na versão de decisão, é NP-completo mesmo quando todas as strings usam o alfabeto fixo

$$\Sigma = \{0, 1\}.$$

Limite do resultado

Para $|\Sigma| = 1$, a resposta depende apenas da menor quantidade do único símbolo entre as strings. Portanto, o alfabeto binário é o menor alfabeto em que a dificuldade pode ocorrer.

Relação com sequências biológicas

Como $\{0, 1\}$ pode ser renomeado para $\{A, C\}$, a dificuldade permanece em uma restrição do alfabeto do DNA; ela não depende de um alfabeto crescente.

Redução que veremos

Blin et al. apresentaram em 2012 uma prova binária alternativa mais direta:

$$\text{INDEPENDENT SET} \propto \text{MULTI-LCS}_{\{0,1\}}.$$

A escolha de vértices define a string $T(I)$

Problema de origem: Independent Set

Dados $G = (V, E)$ e $1 \leq k \leq |V|$, decidir se existe $I \subseteq V$, $|I| \geq k$, tal que nenhuma aresta possui os dois extremos em I .

Seja $V = \{1, \dots, n\}$. Defina os blocos

$$Z = 0^n, \quad B = 0^n 1 = Z1.$$

String associada a um conjunto candidato String-base

Para qualquer $I \subseteq V$, defina

$$T(I) = T_1 T_2 \cdots T_n,$$

$$T_i = \begin{cases} B, & i \in I, \\ Z, & i \notin I. \end{cases}$$

$$S_0 = B^n = (0^n 1)^n.$$

Como $Z \preceq B$, temos $T(I) \preceq S_0$ para qualquer $I \subseteq V$. Além disso,

$$|T(I)| = n^2 + |I|.$$

Assim, B representa um vértice escolhido e Z , um não escolhido.

Limiar da instância de Multi-LCS

Defina $L = n^2 + k$. Então

$$|I| \geq k \quad \Longleftrightarrow \quad |T(I)| \geq L.$$

Os dois blocos Z de S_e codificam a restrição da aresta

Para cada aresta $e = \{u, v\}$, com $u < v$, construa

$$S_e = B^{u-1} Z B^{v-u} Z B^{n-v}, \quad B = 0^n 1, \quad Z = 0^n.$$

Ela possui $n + 1$ blocos.

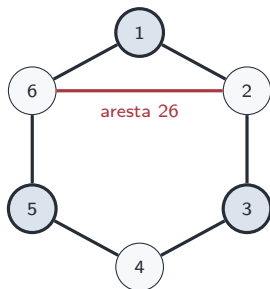
Para obter os n blocos de $T(I)$, pulamos um bloco inteiro de S_e . A tabela mostra alinhamentos suficientes para os três casos:

	B^{u-1}	primeiro Z	B^{v-u}	segundo Z	B^{n-v}
$u \notin I$ $v \in I$	$T_1 \cdots T_{u-1}$	$T_u = Z$	$T_{u+1} \cdots T_v$	pular	$T_{v+1} \cdots T_n$
$u \in I$ $v \notin I$	$T_1 \cdots T_{u-1}$	pular	$T_u \cdots T_{v-1}$	$T_v = Z$	$T_{v+1} \cdots T_n$
$u, v \notin I$	$T_1 \cdots T_{u-1}$	$T_u = Z$	$T_{u+1} \cdots T_{v-1}$ pular um B	$T_v = Z$	$T_{v+1} \cdots T_n$

Obs: $u, v \notin I$

Usamos o primeiro Z como T_u , o segundo como T_v e pulamos um dos blocos B situados entre eles.

A aresta 26 tem os dois extremos fora de I



Em azul: $I = \{1, 3, 5\}$;
em vermelho: a aresta 26.

Instância de Independent Set

$$V = \{1, 2, 3, 4, 5, 6\},$$

$$E = \{12, 23, 34, 45, 56, 16, 26\}.$$

Nenhuma aresta possui os dois extremos em $I = \{1, 3, 5\}$. Na aresta 26, ambos os extremos estão fora de I .

Como $n = 6$, defina $B = 0000001$ e $Z = 000000$. O conjunto I produz

$$T(I) = \underbrace{B}_1 \underbrace{Z}_2 \underbrace{B}_3 \underbrace{Z}_4 \underbrace{B}_5 \underbrace{Z}_6.$$

Comprimento

$$|T(I)| = 6 \cdot 6 + 3 = n^2 + k = 39.$$

$T(I) \preceq S_0 = B^6$: nos vértices pares, removemos o 1 final do bloco.

B marca um vértice escolhido; Z marca um vértice não escolhido.

Todas as strings de aresta admitem o mesmo $T(I)$

Cada S_e possui sete blocos: cinco blocos B e dois blocos Z . Marcamos em **vermelho** o Z alinhado ao vértice fora de I e em **cinza** o bloco pulado.

aresta	string construída	extremo fora de I
12	$Z B Z B B B B$	2
23	$B Z B Z B B B$	2
34	$B B Z B Z B B$	4
45	$B B B Z B Z B$	4
56	$B B B B Z B Z$	6
16	$Z B B B B B Z$	6
26	$B Z B B B B Z$	2 e 6

Como ler a linha da aresta 26

$$S_{26} = B Z B B B B Z.$$

Alinhamos os dois Z a T_2 e T_6 , obtemos $T_4 = Z$ de um bloco B e pulamos o bloco B em cinza. Assim, obtemos $T(I) = B Z B Z B Z$.

Onde aparece a independência?

Toda aresta tem pelo menos um extremo fora de I . Na aresta 26, os dois extremos estão fora: ambos **são representados por Z** . Por isso a mesma string $T(I)$ continua sendo subseqüência de todas as strings construídas.

Conjunto independente $\Rightarrow$ subsequência comum longa

Suponha que $I \subseteq V$ seja independente e $|I| \geq k$. Construa

$$T(I) = T_1 T_2 \cdots T_n, \quad T_i = \begin{cases} B = 0^n 1, & i \in I, \\ Z = 0^n, & i \notin I. \end{cases}$$

$$T(I) \preceq S_0$$

Cada T_i é obtido do bloco $B_i = 0^n 1$ de S_0 mantendo ou removendo seu último símbolo 1.

$$T(I) \preceq S_e \text{ para toda aresta } e = \{u, v\}$$

Como I é independente, $u \notin I$ ou $v \notin I$.

- se $u \notin I$, alinhamos $T_u = Z$ ao primeiro Z e pulamos o segundo;
- se $v \notin I$, pulamos o primeiro Z e alinhamos $T_v = Z$ ao segundo.

Os demais T_i são alinhados aos blocos B restantes; isso funciona também para $T_i = Z$, pois $Z \preceq B$. Se ambos os extremos estão fora de I , podemos ainda usar os dois Z e pular um B , como em S_{26} .

Comprimento preservado

$$|T(I)| = n^2 + |I| \geq n^2 + k = L.$$

Logo, a instância construída de Multi-LCS tem resposta sim.

Uma LCS ótima pode ser colocada na forma $T(J)$

Suponha que a resposta de Multi-LCS seja sim e escolha uma LCS T_{opt} com

$$|T_{\text{opt}}| \geq L = n^2 + k > n^2.$$

Recorde que $S_0 = B_1 B_2 \cdots B_n$, com $B_i = B = 0^n 1$.

Lema de normalização

Sem diminuir o comprimento, podemos escolher T_{opt} de modo que

- todos os n^2 zeros de S_0 sejam preservados;
- os n zeros provenientes de cada B_i permaneçam juntos;
- depois desses zeros, o 1 final de B_i seja preservado ou removido.

Portanto, a LCS normalizada tem a forma

$$T_{\text{opt}} = T(J) = T_1 T_2 \cdots T_n, \quad T_i = \begin{cases} B = 0^n 1, & i \in J, \\ Z = 0^n, & i \notin J. \end{cases}$$

Aqui, J é o conjunto dos vértices cujos blocos preservam o símbolo 1.

Comprimento

Cada bloco fornece n zeros e cada vértice de J fornece mais um símbolo 1:

$$|T(J)| = n^2 + |J|.$$

Logo, $|T(J)| \geq n^2 + k$ implica $|J| \geq k$.

Uma aresta impede escolher simultaneamente seus extremos

Considere uma aresta $e = \{u, v\}$, com $u < v$:

$$S_e = B^{u-1} Z B^{v-u} Z B^{n-v}.$$

Os dois blocos Z ocupam as posições u e $v + 1$. No alinhamento normalizado, os blocos de $T(J)$ usam, em ordem, n dos $n + 1$ blocos de S_e . Portanto, exatamente uma posição r é pulada.

posição pulada	consequência para os extremos da aresta
$r \leq u$	T_v é alinhado ao segundo Z
$u < r < v + 1$	T_u e T_v são alinhados aos dois Z
$r \geq v + 1$	T_u é alinhado ao primeiro Z

Como $Z = 0^n$ não possui o símbolo final 1, em todos os casos

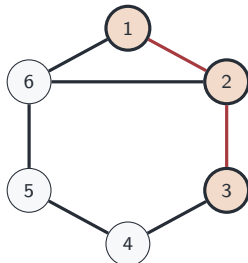
$$T_u = Z \quad \text{ou} \quad T_v = Z.$$

Lema da aresta, em forma contrapositiva

Se $u, v \in J$, então $T(J)$ exige $T_u = T_v = B$ e, portanto, exige o símbolo 1 nos dois extremos. A string S_e impede essa escolha simultânea:

$$\{u, v\} \subseteq J \implies T(J) \not\subseteq S_e.$$

O candidato $J = \{1, 2, 3\}$ não é independente



As arestas 12 e 23 têm
os dois extremos em J .

A mesma instância

$$V = \{1, 2, 3, 4, 5, 6\},$$

$$E = \{12, 23, 34, 45, 56, 16, 26\}.$$

O conjunto $J = \{1, 2, 3\}$ define o candidato

$$T(J) = \underbrace{B}_1 \underbrace{B}_2 \underbrace{B}_3 \underbrace{Z}_4 \underbrace{Z}_5 \underbrace{Z}_6,$$
$$|T(J)| = 36 + 3 = 39.$$

Por que ele deve falhar?

J contém as duas extremidades das arestas 12 e 23. Basta usar a aresta 12: o candidato exige simultaneamente $T_1 = T_2 = B$. O próximo slide mostra por que isso é impossível em S_{12} .

A aresta 12 já elimina o candidato $T(J)$

Para $e = \{1, 2\}$, temos

$$S_{12} = \underbrace{Z}_1 \underbrace{B}_2 \underbrace{Z}_3 \underbrace{B}_4 \underbrace{B}_5 \underbrace{B}_6 \underbrace{B}_7.$$

Para que $T(J)$ fosse subsequência de S_{12} , seus seis blocos teriam de usar seis desses sete blocos, em ordem. Há somente três possibilidades para a posição pulada em relação aos três primeiros blocos:

$$r = 1 \quad \underbrace{\boxed{Z}}_{\text{pulado}} \quad \boxed{B} \quad \boxed{Z} \quad T_1 \leftarrow B, T_2 \leftarrow Z: \text{ falta o 1 de } T_2.$$

$$r = 2 \quad \boxed{Z} \quad \underbrace{\boxed{B}}_{\text{pulado}} \quad \boxed{Z} \quad T_1 \leftarrow Z, T_2 \leftarrow Z: \text{ faltam os dois 1s.}$$

$$r \geq 3 \quad \boxed{Z} \quad \boxed{B} \quad \underbrace{\boxed{\dots}}_{\text{pulado depois}} \quad T_1 \leftarrow Z, T_2 \leftarrow B: \text{ falta o 1 de } T_1.$$

Como $1, 2 \in J$ exige $T_1 = T_2 = B$, nenhuma possibilidade funciona:

$$T(\{1, 2, 3\}) = B B B Z Z Z \not\preceq S_{12}.$$

A contrapositiva conclui a implicação de volta

Considere qualquer conjunto $J \subseteq V$ e a string associada $T(J)$.

Contrapositiva

Se J não é independente, existe uma aresta $e = \{u, v\}$ com $u, v \in J$. Pelo lema da aresta,

$$\{u, v\} \subseteq J \implies T(J) \not\subseteq S_e.$$

Consequentemente, $T(J)$ não é subsequência comum das strings construídas:

$$J \text{ não é independente} \implies T(J) \text{ não é comum.}$$

A afirmação contrapositiva equivalente é

$$T(J) \text{ é comum} \implies J \text{ é independente.}$$

Aplicação à LCS normalizada

Da resposta sim de Multi-LCS, obtemos uma LCS normalizada $T(J)$ que é comum e satisfaz

$$|T(J)| = n^2 + |J| \geq n^2 + k.$$

Logo, J é independente e $|J| \geq k$.

A redução binária conclui a NP-completude

Tamanho da transformação

Para um grafo com n vértices e m arestas, construímos

- $m + 1$ strings;
- $|S_0| = n^2 + n$;
- $|S_e| = n^2 + 2n - 1 = (n + 1)^2 - 2$ para cada $e \in E$.

O tamanho total é $O(mn^2)$; a transformação é polinomial.

Equivalência

G possui conjunto independente de tamanho $\geq k$



$\exists z : |z| \geq n^2 + k$ e $z \preceq S_i$ para toda string construída.

NP-dificuldade

INDEPENDENT SET $\propto$ MULTI-LCS $_{\{0,1\}}$.

NP-completude

Como a decisão também pertence a NP, Multi-LCS é NP-completo mesmo sobre $\{0, 1\}$.

A versão de otimização correspondente é NP-difícil.

Em bioinformática, sinais e operações também importam

Tema	Formulação tratável	Formulação difícil
rearranjos	distância por inversões entre permutações com sinais: polinomial	distância por inversões sem sinais: NP-difícil
DCJ	distância entre dois genomas com marcadores únicos: polinomial	mediana DCJ de três genomas: NP-difícil
filogenia	parcimônia em uma árvore fixa: polinomial	escolher uma árvore de parcimônia ótima: NP-difícil em formulações clássicas
folding de RNA	estruturas secundárias sem pseudonós em modelos usuais: programação dinâmica	modelos gerais de energia com pseudonós: NP-completos

Restringir ou estender o modelo exige nova análise

Restrições

Se um problema geral está em P, qualquer caso restrito também está em P.
Mas um caso restrito pode ser fácil mesmo quando o problema geral é difícil.

Extensões

Se uma extensão contém um problema NP-difícil como caso particular, a extensão também é NP-difícil.

Não vale a recíproca automática

A dificuldade de uma formulação geral não prova que toda variante particular seja difícil; a facilidade de uma variante não prova que a extensão continue fácil.

Um artigo de complexidade deve responder perguntas precisas

Ao ler os artigos dos trabalhos, identifique:

1. qual é a instância e como seu tamanho é medido;
2. qual é a versão de decisão e qual é o limiar k ;
3. quais operações, custos e restrições são permitidos;
4. quais quantidades são fixas e quais fazem parte da entrada;
5. qual é o certificado e por que a verificação é polinomial;
6. qual é o problema de origem e a direção da redução;
7. se o resultado é NP-difícil ou NP-completo;
8. qual aspecto biológico foi preservado pela abstração.